

Selected Labs in Software Engineering (Projects)

"Choose one of the projects from the list below, then implement and design a GUI Java application using design patterns. Please note that both the Singleton pattern and the Factory pattern are mandatory for all projects. Additionally, you must incorporate at least three other design patterns.

- You must upload the code folder (exported from NetBeans).
- You must also upload project documentation, which should include descriptions for each class and pattern used. Justify the use of each pattern in your project.
- The code must be thoroughly commented.
- Note: The grades for each team member will vary based on their individual contributions to the discussion and the project. Each member's grade will reflect their level of involvement, effort, and the quality of their input. As a result, team members may receive different grades."

1. Library Management System

Description: A desktop application for managing library books, users, and transactions (borrow/return).

- **Singleton Pattern:**
 - *Task:* Use Singleton for managing the **Database Connection**. The database connection should be a single instance throughout the application lifecycle.
 - *Task:* A **Logger** class to handle application-wide logging can be a Singleton to ensure consistent log management.
- **Factory Pattern:**
 - *Task:* Create a **Factory for Different Types of Books** (e.g., Software Engineering, Management, Artificial Intelligence). The Factory can instantiate objects of different book categories based on input.
 - *Task:* A **Factory for User Creation** (e.g., Admin, Regular User) to ensure that user creation follows a predefined structure.

Patterns Used: Singleton, Factory

2. Hotel Reservation System

Description: An application for booking hotel rooms, managing reservations, and customer services.

- **Singleton Pattern:**
 - *Task:* A **Reservation Manager** can be implemented as a Singleton to manage all reservation activities across the system.
 - *Task:* Use Singleton for **Payment Processor** to ensure there's only one instance managing payments and processing them.
- **Factory Pattern:**
 - *Task:* A **Factory for Room Types** (e.g., Standard, Deluxe, Suite) to create room objects dynamically based on user choice.
 - *Task:* A **Factory for Customer Profile Creation** to generate objects representing customers, such as regular, VIP, or corporate clients.

Patterns Used: Singleton, Factory

3. Employee Management System

Description: A system for managing employee records, attendance, payroll, and more.

- **Singleton Pattern:**
 - *Task:* Use Singleton for the **Payroll System** to ensure that the payroll processing is centralized and consistent.
 - *Task:* A **Database Connection Manager** to ensure only one instance is used to manage all employee-related data.
- **Factory Pattern:**
 - *Task:* A **Factory for Employee Types** (e.g., Full-time, Part-time, Contractor) to create different employee objects with appropriate attributes.
 - *Task:* A **Factory for Department Creation** to instantiate objects related to different departments, such as HR, Finance, or IT.

Patterns Used: Singleton, Factory

4. Inventory Management System

Description: A desktop application for managing products, stock levels, suppliers, and orders.

- **Singleton Pattern:**
 - *Task:* A **Stock Management System** can be a Singleton to ensure inventory updates are tracked across the system consistently.
 - *Task:* Use Singleton for **Reporting Module** to ensure reports (e.g., sales reports, inventory status) are generated from a single instance.
- **Factory Pattern:**
 - *Task:* A **Factory for Product Categories** (e.g., Electronics, Furniture, Groceries) to dynamically create product objects based on the category selected.
 - *Task:* A **Factory for Supplier Types** (e.g., Local, International) to create supplier objects with specific properties.

Patterns Used: Singleton, Factory

5. Banking System

Description: A desktop application for managing bank accounts, transactions, customer profiles, and loans.

- **Singleton Pattern:**
 - *Task:* A **Transaction Manager** class can be a Singleton to ensure only one instance handles all the financial transactions.
 - *Task:* **Authentication and Session Management** should be a Singleton to maintain consistency in user sessions.
- **Factory Pattern:**
 - *Task:* A **Factory for Account Types** (e.g., Savings, Checking, Loan) to create different account objects based on the user's choice.
 - *Task:* A **Factory for Loan Types** (e.g., Home, Personal, Car) to create objects representing different loan types.

Patterns Used: Singleton, Factory

6. E-commerce System

Description: A desktop application for managing product listings, customers, orders, and payments.

- **Singleton Pattern:**
 - *Task:* A **Cart Manager** can be a Singleton to handle the shopping cart data across the entire user session.
 - *Task:* Use Singleton for the **Payment Gateway** to ensure consistent processing of payments (credit card, PayPal, etc.).
- **Factory Pattern:**

- *Task:* A **Factory for Product Types** (e.g., Electronics, Clothing, Home Appliances) to create products based on category.
- *Task:* A **Factory for Order Processing** (e.g., Standard, Express) to create different order handling processes based on the customer's choice.

Patterns Used: Singleton, Factory

7. Task Management System

Description: A desktop app for managing tasks, deadlines, and team collaboration.

- **Singleton Pattern:**
 - *Task:* A **Task Manager** can be a Singleton to ensure all task data is managed through a single point.
 - *Task:* Use Singleton for **Notification System** to ensure consistent notification delivery across all tasks.
- **Factory Pattern:**
 - *Task:* A **Factory for Task Types** (e.g., Bug, Feature, Improvement) to create different task objects based on the user's input.
 - *Task:* A **Factory for User Roles** (e.g., Admin, Developer, Tester) to create role-specific user objects for task assignments.

Patterns Used: Singleton, Factory

8. Student Management System

Description: An application for managing student records, grades, courses, and timetables.

- **Singleton Pattern:**
 - *Task:* A **Course Registration System** can be a Singleton to track students course enrollment across the system.
 - *Task:* A **Grade Processing System** should be a Singleton to ensure grade calculations are handled uniformly.
- **Factory Pattern:**
 - *Task:* A **Factory for Course Types** (e.g., Core, Elective, Lab) to create course objects dynamically.
 - *Task:* A **Factory for Student Types** (e.g., Undergraduate, Graduate, Part-time) to create different student profiles.

Patterns Used: Singleton, Factory

9. Document Management System

Description: A desktop application for storing and managing digital documents (PDF, Word, Excel, etc.).

- **Singleton Pattern:**
 - *Task:* A **Document Repository** can be a Singleton to ensure only one instance manages all document storage and retrieval.
 - *Task:* Use Singleton for **User Access Control** to manage permissions and roles consistently.
- **Factory Pattern:**
 - *Task:* A **Factory for Document Types** (e.g., PDF, Word, Excel) to create document objects depending on file type.
 - *Task:* A **Factory for User Roles** (e.g., Admin, Viewer, Editor) to create objects representing different levels of access to documents.

Patterns Used: Singleton, Factory

10. Movie Ticket Booking System

Description: A system for booking movie tickets, managing showtimes, theaters, and user preferences

- **Singleton Pattern:**
 - *Task:* Use Singleton for **Ticket Booking System** to ensure all ticket bookings are handled by one centralized system.
 - *Task:* A **Session Manager** can be a Singleton to handle customer sessions and booking status.
- **Factory Pattern:**
 - *Task:* A **Factory for Movie Types** (e.g., Action, Comedy, Drama) to create movie objects based on the selected genre.
 - *Task:* A **Factory for Theater Locations** (e.g., Cinema Hall, IMAX) to create theater-related objects.

Patterns Used: Singleton, Factory

11. Restaurant Management System

Description: A system for managing restaurant orders, menu items, reservations, and customer details.

- **Singleton Pattern:**
 - *Task:* **Order Manager** - A Singleton to handle all order operations, ensuring there's one centralized point for order processing.
 - *Task:* **Payment System** - A Singleton to manage payment processing, ensuring consistent payment logic throughout the app.
- **Factory Pattern:**
 - *Task:* **Factory for Menu Item Types** (e.g., Appetizers, Main Courses, Desserts) to create different types of menu items.
 - *Task:* **Factory for Table Types** (e.g., Regular, VIP, Outdoor) to create table objects depending on the user's selection.

Patterns Used: Singleton, Factory

12. Medical Clinic Management System

Description: A desktop application for managing patient records, doctor schedules, appointments, and medical histories.

- **Singleton Pattern:**
 - *Task:* **Patient Database Manager** - A Singleton class for managing patient data, ensuring consistent access and updates.
 - *Task:* **Appointment Scheduling System** - A Singleton to ensure that appointment slots are checked and assigned consistently across all users.
- **Factory Pattern:**
 - *Task:* **Factory for Medical Records** (e.g., Patient History, Prescriptions, Lab Results) to create different medical record types.
 - *Task:* **Factory for Doctor Specializations** (e.g., Cardiologist, Neurologist, General Practitioner) to instantiate doctor objects based on specialty.

Patterns Used: Singleton, Factory

13. Point of Sale (POS) System

Description: A system for managing sales transactions, product inventory, and customer information for retail stores.

- **Singleton Pattern:**
 - *Task:* **Sales Transaction Manager** - A Singleton that tracks and processes all sales transactions across the system.
 - *Task:* **Inventory Manager** - A Singleton that manages stock levels, inventory updates, and low-stock alerts.
- **Factory Pattern:**

- **Task: Factory for Product Categories** (e.g., Electronics, Apparel, Groceries) to create product objects based on the category.
- **Task: Factory for Payment Methods** (e.g., Credit Card, Cash, Mobile Payment) to create different payment processors.

Patterns Used: Singleton, Factory

14. Weather Monitoring System

Description: A system for tracking and displaying weather data, including temperature, humidity, and forecasts.

- **Singleton Pattern:**
 - **Task: Weather Data Fetcher** - A Singleton that handles the fetching of weather data from external APIs or databases.
 - **Task: User Preferences Manager** - A Singleton to manage user preferences such as units of measurement (Celsius/Fahrenheit, km/h, mph).
- **Factory Pattern:**
 - **Task: Factory for Weather Data Types** (e.g., Temperature, Humidity, Wind Speed) to create weather data objects based on the type.
 - **Task: Factory for Weather Alerts** (e.g., Storm, Snow, Heatwave) to generate weather alerts for specific conditions.

Patterns Used: Singleton, Factory

15. Chat Application

Description: A desktop chat application allowing users to send messages, create groups, and manage contacts.

- **Singleton Pattern:**
 - **Task: Chat Session Manager** - A Singleton to manage active chat sessions, ensuring only one instance tracks the chat history and status.
 - **Task: User Authentication System** - A Singleton to handle user login and session validation across multiple chat windows.
- **Factory Pattern:**
 - **Task: Factory for Message Types** (e.g., Text, Image, Video) to create message objects based on the content type.
 - **Task: Factory for Group Types** (e.g., Public, Private, Admin) to create different group objects with specific rules.

Patterns Used: Singleton, Factory

16. Travel Management System

Description: A desktop application to help travel agencies manage flight bookings, hotel reservations, and customer preferences.

- **Singleton Pattern:**
 - **Task: Booking Manager** - A Singleton to ensure booking operations are handled consistently (flights, hotels, packages).
 - **Task: User Profile Manager** - A Singleton to handle user details such as preferences, loyalty points, and booking history.
- **Factory Pattern:**
 - **Task: Factory for Travel Packages** (e.g., Luxury, Adventure, Cultural) to create package objects based on user choice.
 - **Task: Factory for Accommodation Types** (e.g., Hotel, Hostel, Resort) to create accommodation objects depending on the user's selection.

Patterns Used: Singleton, Factory

17. News Aggregator System

Description: A system to fetch, categorize, and display news articles from various sources, including search functionality.

- **Singleton Pattern:**
 - *Task: News Data Fetcher* - A Singleton to fetch news articles from various sources and APIs, ensuring a single instance of the data retrieval process.
 - *Task: User Settings Manager* - A Singleton to store user preferences for news sources, categories, and display settings.
- **Factory Pattern:**
 - *Task: Factory for News Categories* (e.g., Politics, Technology, Sports) to generate news category objects.
 - *Task: Factory for News Article Types* (e.g., Blog, Article, Video) to create objects based on content type.

Patterns Used: Singleton, Factory

18. File Compression Utility

Description: A desktop application for compressing and decompressing files (e.g., ZIP, TAR, RAR).

- **Singleton Pattern:**
 - *Task: Compression Manager* - A Singleton to handle the compression and decompression processes, ensuring one point of control.
 - *Task: File Handler* - A Singleton that manages file input/output operations for compression and extraction.
- **Factory Pattern:**
 - *Task: Factory for Compression Algorithms* (e.g., ZIP, TAR, RAR) to create objects based on the selected compression type.
 - *Task: Factory for File Types* (e.g., Text, Images, Videos) to handle files differently based on their types.

Patterns Used: Singleton, Factory

19. Online Quiz System

Description: A desktop application for creating, taking, and scoring quizzes.

- **Singleton Pattern:**
 - *Task: Quiz Manager* - A Singleton class that manages the creation, start, and progress of quizzes.
 - *Task: Score Manager* - A Singleton to handle scoring and storing results for quizzes.
- **Factory Pattern:**
 - *Task: Factory for Question Types* (e.g., Multiple Choice, True/False, Short Answer) to generate questions based on the quiz setup.
 - *Task: Factory for User Roles* (e.g., Admin, Student, Teacher) to create user profiles and assign access levels.

Patterns Used: Singleton, Factory

20. Fitness Tracker Application

Description: A system for tracking fitness activities, goals, and progress (e.g., workouts, nutrition).

- **Singleton Pattern:**
 - *Task: Activity Tracker* - A Singleton to track user activities like running, cycling, or gym workouts.
 - *Task: Goal Management System* - A Singleton to store and manage user fitness goals (e.g., weight loss, strength training).
- **Factory Pattern:**

- *Task: **Factory for Exercise Types*** (e.g., Cardio, Strength, Flexibility) to create different exercise objects based on the workout.
- *Task: **Factory for Nutrition Plans*** (e.g., Weight Loss, Muscle Gain) to create specific diet plans based on the user's goal.

Patterns Used: Singleton, Factory

21. Document Collaboration System

Description: A desktop application allowing multiple users to collaborate on documents in real time.

- **Singleton Pattern:**
 - *Task: **Document Locking System*** - A Singleton to manage document editing locks and prevent concurrent modifications.
 - *Task: **Real-time Update Manager*** - A Singleton to handle real-time updates and synchronize document changes across users.
- **Factory Pattern:**
 - *Task: **Factory for Document Types*** (e.g., Word, Excel, PDF) to create document objects based on the file type.
 - *Task: **Factory for User Roles*** (e.g., Admin, Editor, Viewer) to create role-specific objects for access control.

Patterns Used: Singleton, Factory